

```
#import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
data1 = pd.read_csv('/content/TTTTH.csv')
print(data1)
```

```

30  738.2072032  17.01910811
31  1604.165889  1.631503985
32  552.2991104  8.232019162
33  1846.14446  9.422087847
34  1846.324159  9.643113644
35  864.5255923  8.887114363
36  864.5412998  9.147054081
37  628.5169298  13.44922343
38  961.0243161 -0.181818182
39      #DIV/0!      #DIV/0!

```

```

prt = data1['Profit']
amount = data1['Amount']

```

✓ Correlation Analysis

First, I will ensure the 'Profit' and 'Amount' columns are numeric, coercing any errors to `NaN`. Then, I will calculate the correlation between them.

```

data1['Profit'] = pd.to_numeric(data1['Profit'], errors='coerce')
data1['Amount'] = pd.to_numeric(data1['Amount'], errors='coerce')

# Calculate correlation between 'Profit' and 'Amount'
correlation_profit_amount = data1['Profit'].corr(data1['Amount'])
print(f"Correlation between Profit and Amount: {correlation_profit_amount:.4f}")

```

Correlation between Profit and Amount: 0.5353

Now, let's look at the correlation matrix for all numeric columns in the dataset to understand the relationships between different variables.

```

numeric_df = data1.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()
print("Correlation Matrix of Numeric Columns:")
display(correlation_matrix)

```

Correlation Matrix of Numeric Columns:

	Amount	Profit	Quantity	Unnamed: 12	Unnamed: 13	Unnamed: 14
Amount	1.000000	5.353258e-01	-7.544092e-03	NaN	0.896927	NaN
Profit	0.535326	1.000000e+00	2.387561e-07	NaN	-0.241176	NaN
Quantity	-0.007544	2.387561e-07	1.000000e+00	NaN	-0.329687	NaN
Unnamed: 12	NaN	NaN	NaN	NaN	NaN	NaN

Next steps: [Generate code with correlation_matrix](#)

[New interactive sheet](#)

✓ Regression Analysis

Now, let's build a regression model to understand how 'Profit' is influenced by other numerical variables like 'Amount' and 'Quantity'. We'll use a simple Linear Regression model for this.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
```

First, we need to prepare our data. We'll define 'Profit' as our target variable (y) and 'Amount' and 'Quantity' as our features (X). We'll also drop any rows with missing values that might have been introduced during the conversion to numeric types.

```
# Drop rows with NaN values in the relevant columns for regression
df_cleaned = data1.dropna(subset=['Profit', 'Amount', 'Quantity']).copy()

# Define features (X) and target (y)
X = df_cleaned[['Amount', 'Quantity']]
y = df_cleaned['Profit']

print(f"Shape of features (X): {X.shape}")
print(f"Shape of target (y): {y.shape}")

Shape of features (X): (40, 2)
Shape of target (y): (40,)
```

Next, we'll split our dataset into training and testing sets. This allows us to train the model on one portion of the data and evaluate its performance on unseen data, preventing overfitting.

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")

Training set size: 32 samples
Testing set size: 8 samples
```

Now, we will initialize and train a Linear Regression model. This model will learn the relationship between 'Amount', 'Quantity', and 'Profit' from the training data.

```
# Initialize the Linear Regression model
model = LinearRegression()

# Train the model using the training data
model.fit(X_train, y_train)
```

```
model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
print(f"Model coefficients: {model.coef_}")
print(f"Model intercept: {model.intercept_}")
```

```
Linear Regression model trained successfully.
Model coefficients: [ 0.17582573 15.06763826]
Model intercept: 39.2908164894875
```

After training, we evaluate the model's performance on the test set. We'll use common metrics like Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2).

```
# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared ( $R^2$ ): {r2:.2f}")
```

```
Mean Squared Error (MSE): 655821.84
Root Mean Squared Error (RMSE): 809.83
R-squared ( $R^2$ ): -0.03
```

Finally, let's visualize the actual vs. predicted 'Profit' values to get a better sense of how well our model is performing.

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2) # Diagonal line
plt.xlabel('Actual Profit')
plt.ylabel('Predicted Profit')
plt.title('Actual vs. Predicted Profit')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



✓ Random Forest Regressor Model

Let's try a more sophisticated model: the Random Forest Regressor. This ensemble model can capture non-linear relationships and interactions better than simple linear regression.

```
from sklearn.ensemble import RandomForestRegressor

# We will reuse the already cleaned and split data (X_train, X_test, y_train,
# If you wish to re-define X and y, you can do so here:
# df_cleaned = data1.dropna(subset=['Profit', 'Amount', 'Quantity']).copy()
# X = df_cleaned[['Amount', 'Quantity']]
# y = df_cleaned['Profit']
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ra
```

Now, let's initialize and train the Random Forest Regressor model. We'll start with default parameters, but these can be tuned for better performance.

```
# Initialize the Random Forest Regressor model
rf_model = RandomForestRegressor(random_state=42)

# Train the model
rf_model.fit(X_train, y_train)

print("Random Forest Regressor model trained successfully.")
```

Random Forest Regressor model trained successfully.

Next, we'll evaluate the Random Forest model's performance on the test set using the same metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared (R^2).

```
# Make predictions on the test set
rf_y_pred = rf_model.predict(X_test)

# Evaluate the model
rf_mse = mean_squared_error(y_test, rf_y_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_y_pred)

print(f"Random Forest - Mean Squared Error (MSE): {rf_mse:.2f}")
print(f"Random Forest - Root Mean Squared Error (RMSE): {rf_rmse:.2f}")
print(f"Random Forest - R-squared ( $R^2$ ): {rf_r2:.2f}")
```

Random Forest - Mean Squared Error (MSE): 503531.29
Random Forest - Root Mean Squared Error (RMSE): 709.60
Random Forest - R-squared (R^2): 0.21

Finally, let's visualize the actual vs. predicted 'Profit' values for the Random Forest Regressor to compare its performance against the linear model.

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, rf_y_pred, alpha=0.7)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2) # Diagonal line
plt.xlabel('Actual Profit')
plt.ylabel('Predicted Profit')
plt.title('Random Forest: Actual vs. Predicted Profit')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Random Forest: Actual vs. Predicted Profit

